3

RLHF

PROBLEMAS DE LOS LLM

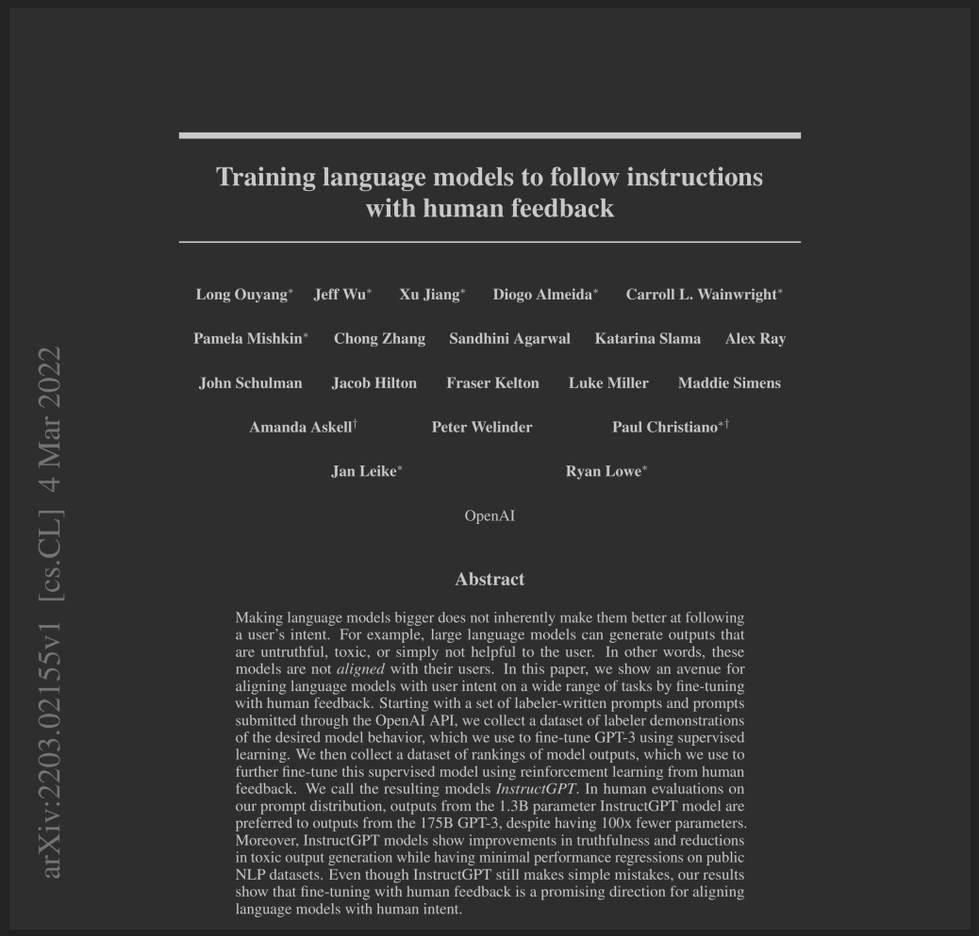
- Los modelos que solo predecían la siguiente palabra a menudo:
  - ✓ Se desviaban.
  - ✓ Inventaban cosas.
  - ✓ Generaban respuestas problemáticas.

PROBLEMAS DE LOS LLM

- Para resolver estos problemas se buscó alinearlos, esto es, que hagan lo que el usuario busca, es decir el objetivo era que:
  - ✓ Fueran útiles.
  - ✓ Ayudaran a resolver tareas.
  - ✓ Fueran honestos (que no fabriquen información).
  - ✓ Fueran inofensivos (que no causaran daño).

INTRODUCCIÓN

- RLHF (Reinforcement Learning from Human Feedback) es un método para afinar LLMs después de que han sido entrenados.
- El artículo que lo describe se titula "Entrenamiento de modelos de lenguaje para seguir instrucciones con retroalimentación humana" (Ouyang et al., 2022).



Training language models to follow instructions with human feedback

Long Ouyang* Jeff Wu* Xu Jiang* Diogo Almeida* Carroll L. Wainwright*
Pamela Mishkin* Chong Zhang Sandhini Agarwal Katarina Slama Alex Ray
John Schulman Jacob Hilton Fraser Kelton Luke Miller Maddie Simens
Amanda Askell† Peter Welinder Paul Christiano*†
Jan Leike* Ryan Lowe*
OpenAI

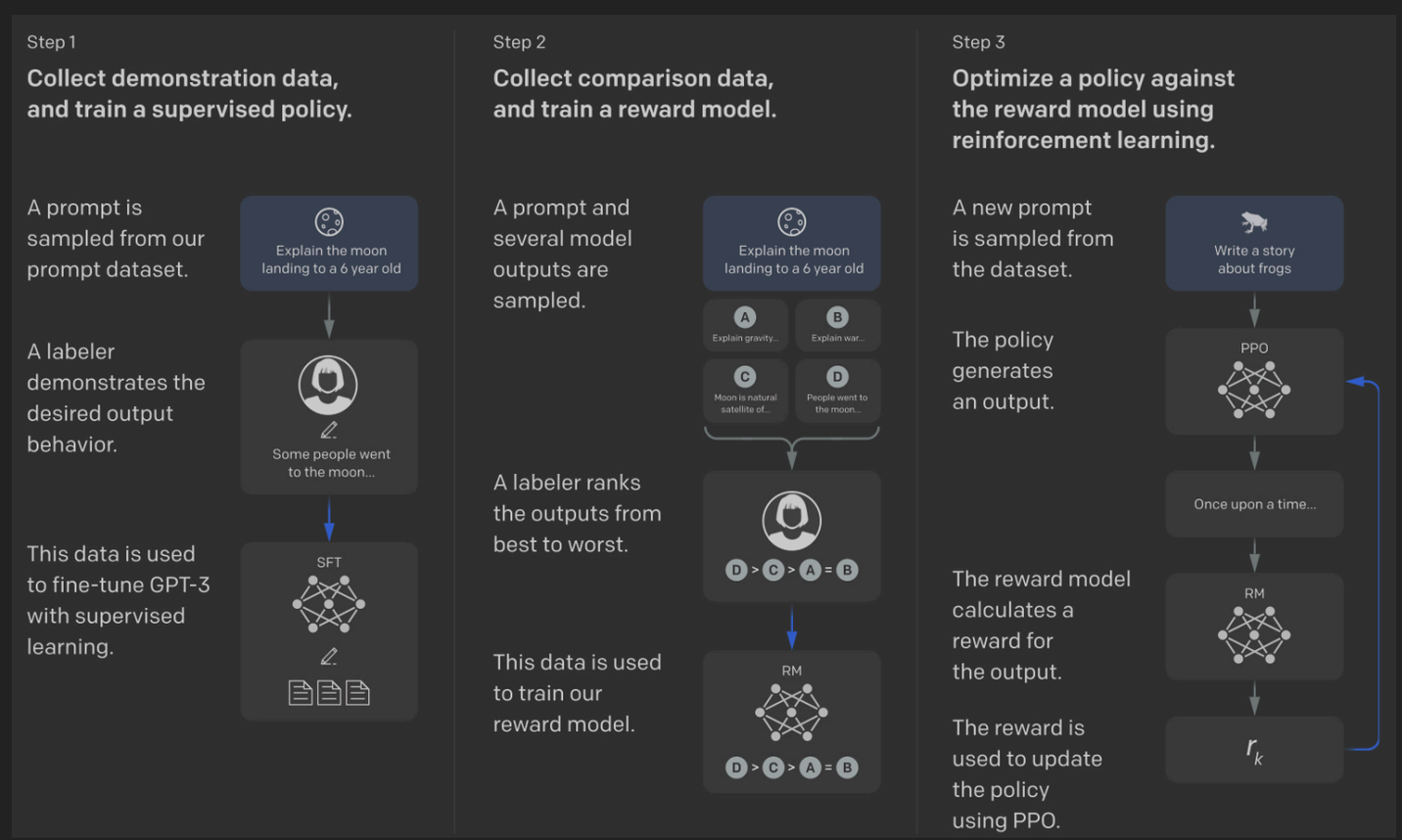
arXiv:2203.02155v1 [cs.CL] 4 Mar 2022

Abstract

Making language models bigger does not inherently make them better at following a user's intent. For example, large language models can generate outputs that are untruthful, toxic, or simply not helpful to the user. In other words, these models are not *aligned* with their users. In this paper, we show an avenue for aligning language models with user intent on a wide range of tasks by fine-tuning with human feedback. Starting with a set of labeler-written prompts and prompts submitted through the OpenAI API, we collect a dataset of labeler demonstrations of the desired model behavior, which we use to fine-tune GPT-3 using supervised learning. We then collect a dataset of rankings of model outputs, which we use to further fine-tune this supervised model using reinforcement learning from human feedback. We call the resulting models *InstructGPT*. In human evaluations on our prompt distribution, outputs from the 1.3B parameter InstructGPT model are preferred to outputs from the 175B GPT-3, despite having 100x fewer parameters. Moreover, InstructGPT models show improvements in truthfulness and reductions in toxic output generation while having minimal performance regressions on public NLP datasets. Even though InstructGPT still makes simple mistakes, our results show that fine-tuning with human feedback is a promising direction for aligning language models with human intent.

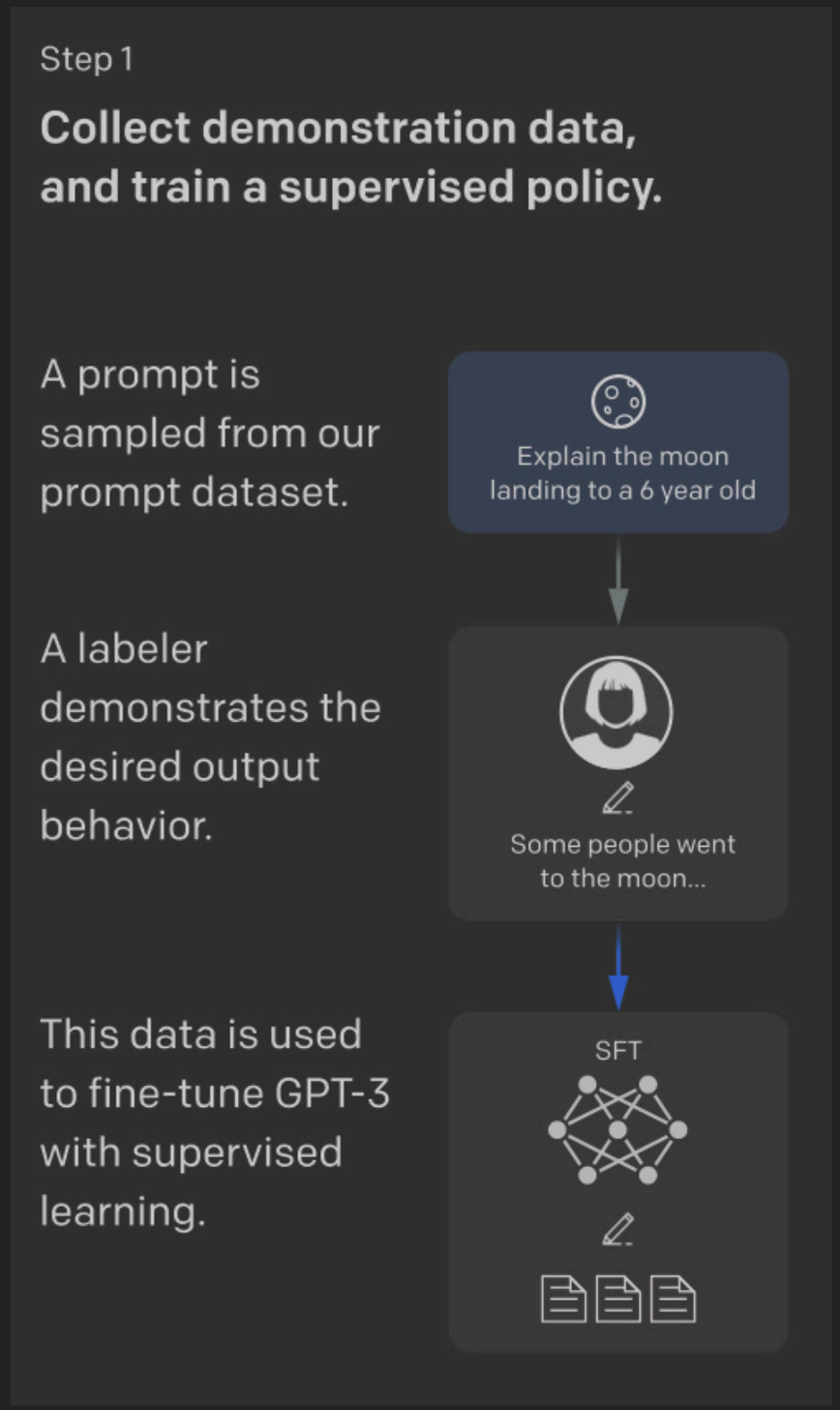
CICLO DE VIDA DE RLHF

- El método RLHF consta de 3 pasos:
  - ✓ Paso 1. Ajuste fino supervisado (supervised fine-tuning, SFT).
  - ✓ Paso 2. Entrenar un modelo de recompensa (reward model, RM) con rankings humanos.
  - ✓ Paso 3. Optimizar el LLM con PPO usando el RM.



PASO 1 – AJUSTE FINO SUPERVISADO

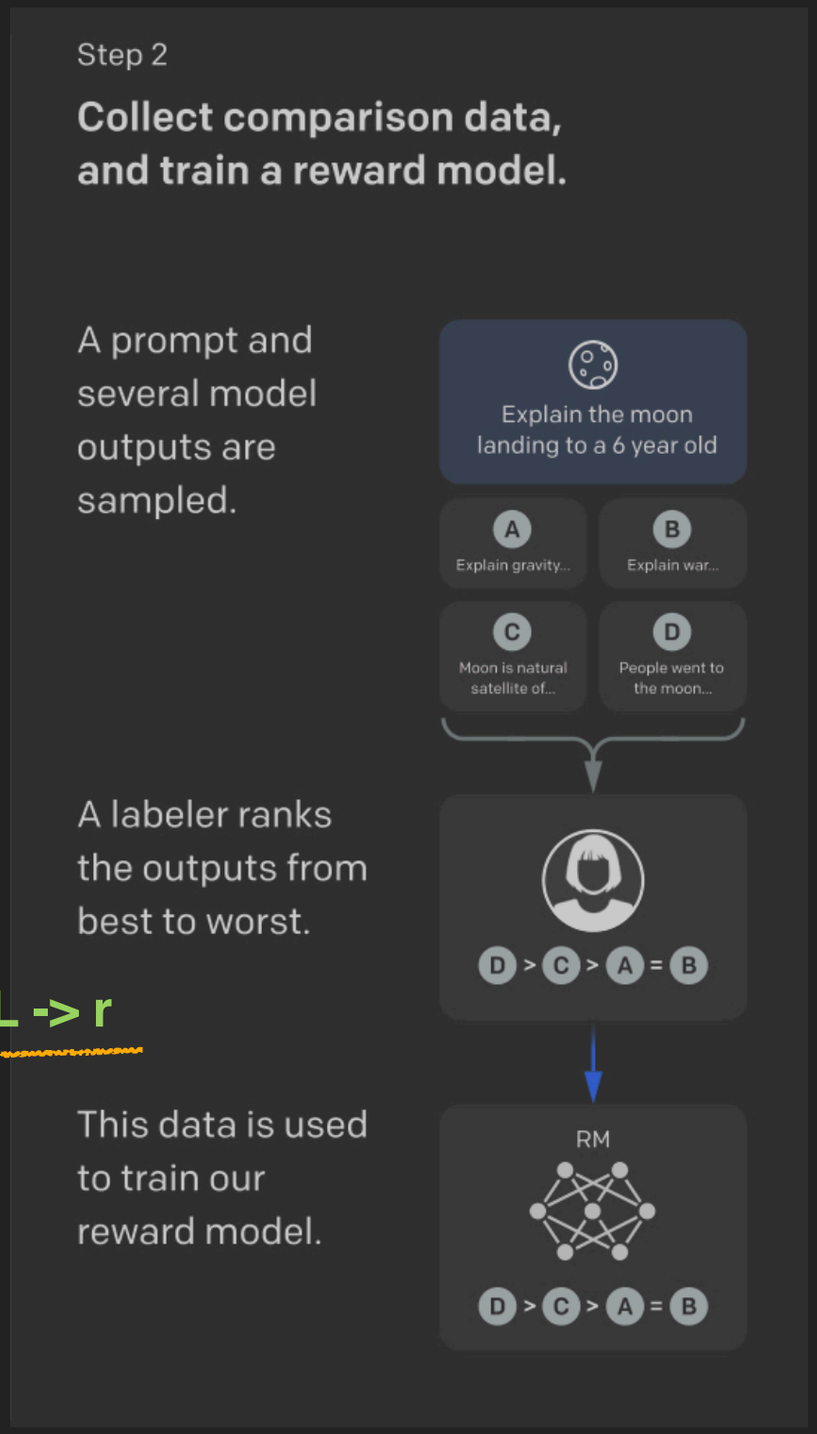
- Se usa un modelo que ya esta entrenado (Ej. GPT-3, Generative Pre-trained Transformer) (Vaswani et al, 2017) y se lo ajusta usando ejemplos escritos por humanos que muestran cómo seguir instrucciones especificas.
- ¿Qué se hace?
  - ✓ Se toma una pregunta (prompt) desde un dataset. Ejemplo: "Explica el alunizaje en forma simple".
  - ✓ Un humano (etiquetador) escribe una respuesta ideal o deseada.
  - ✓ Esta colección de ejemplos sirve para entrenar un modelo (como GPT-3) usando aprendizaje supervisado, lo que se conoce como SFT (Supervised Fine-Tuning).
- ¿Cuál es el objetivo?
  - ✓ Enseñar al modelo a generar respuestas similares a las que haría un humano.



PASO 2 – ENTRENAR UN MODELO DE RECOMPENSA

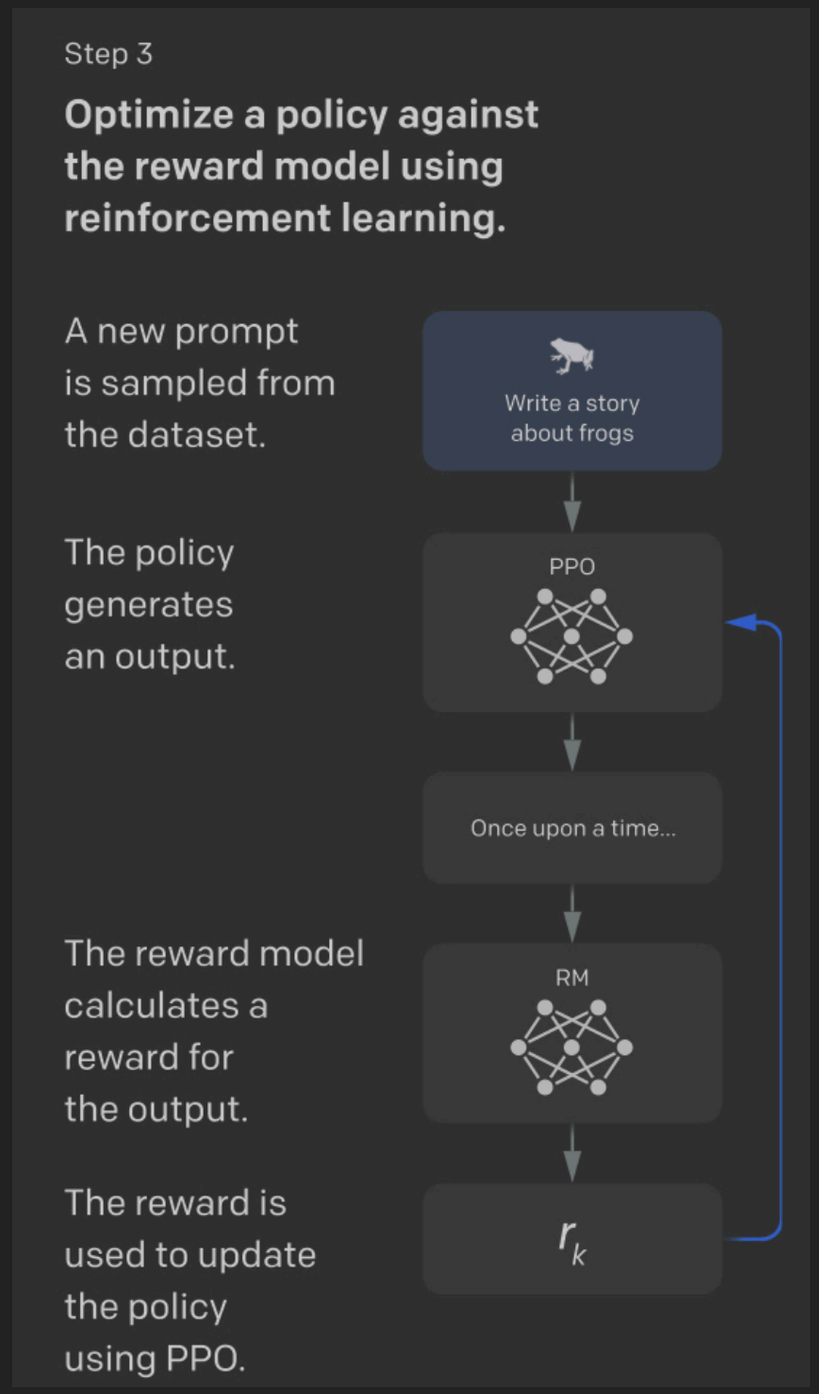
- Se muestra a humanos pares de respuesta que generó el modelo anterior en donde los humanos dicen "prefiero esta respuesta". Con las respuestas se entrena un modelo llamado RM que aprende a predecir qué respuesta van a preferir los humanos.
- ¿Qué se hace?
  - ✓ Se toma un prompt (el mismo del paso 1 o similar) y se generan varias respuestas diferentes con el modelo entrenado en el Paso 1.
  - ✓ Un humano etiqueta estas respuestas, ordenándolas de mejor a peor.
  - ✓ Esta información se utiliza para entrenar un modelo de recompensa (Reward Model, RM), el cual aprende a asignar puntuaciones a las respuestas según qué tanto agradan a los humanos.
- ¿Cuál es el objetivo?
  - ✓ Tener un modelo que pueda evaluar automáticamente la calidad de una respuesta, como lo haría un humano.

(prompt1, R1', R2', … Rn') → RM → L -> r



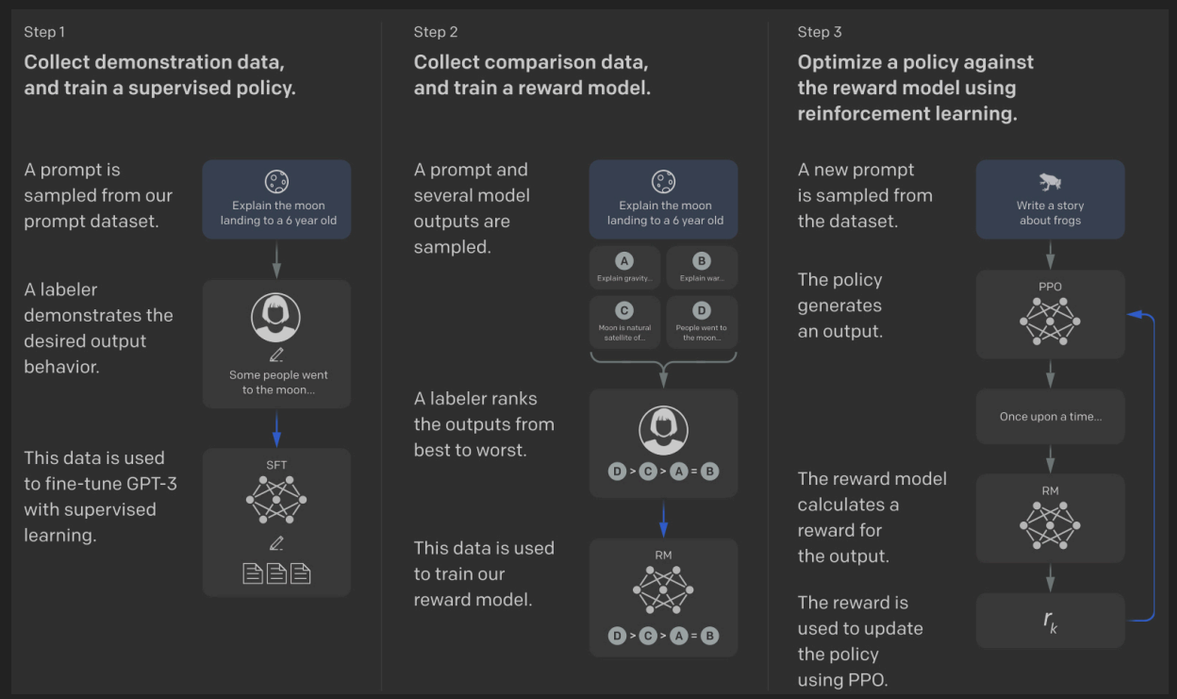
PASO 3 – OPTIMIZAR EL LLM CON PPO

- Luego se usa ese RM para seguir entrenando al modelo SFT pero con RL. El RM guía al SFT para que genere respuestas que le resulten mejor a los humanos (que maximicen esa puntuación de preferencia).
- ¿Qué se hace?
  - ✓ Se toma un nuevo prompt y el modelo genera una respuesta.
  - ✓ Esta respuesta es evaluada por el modelo de recompensa, que le asigna un valor de recompensa (r_k).
  - ✓ Esa recompensa se utiliza en un algoritmo de RL (PPO) para ajustar el modelo, premiando las buenas respuestas y penalizando las malas.
- ¿Cuál es el objetivo?
  - ✓ Mejorar continuamente el modelo para que genere respuestas cada vez más alineadas con las preferencias humanas.



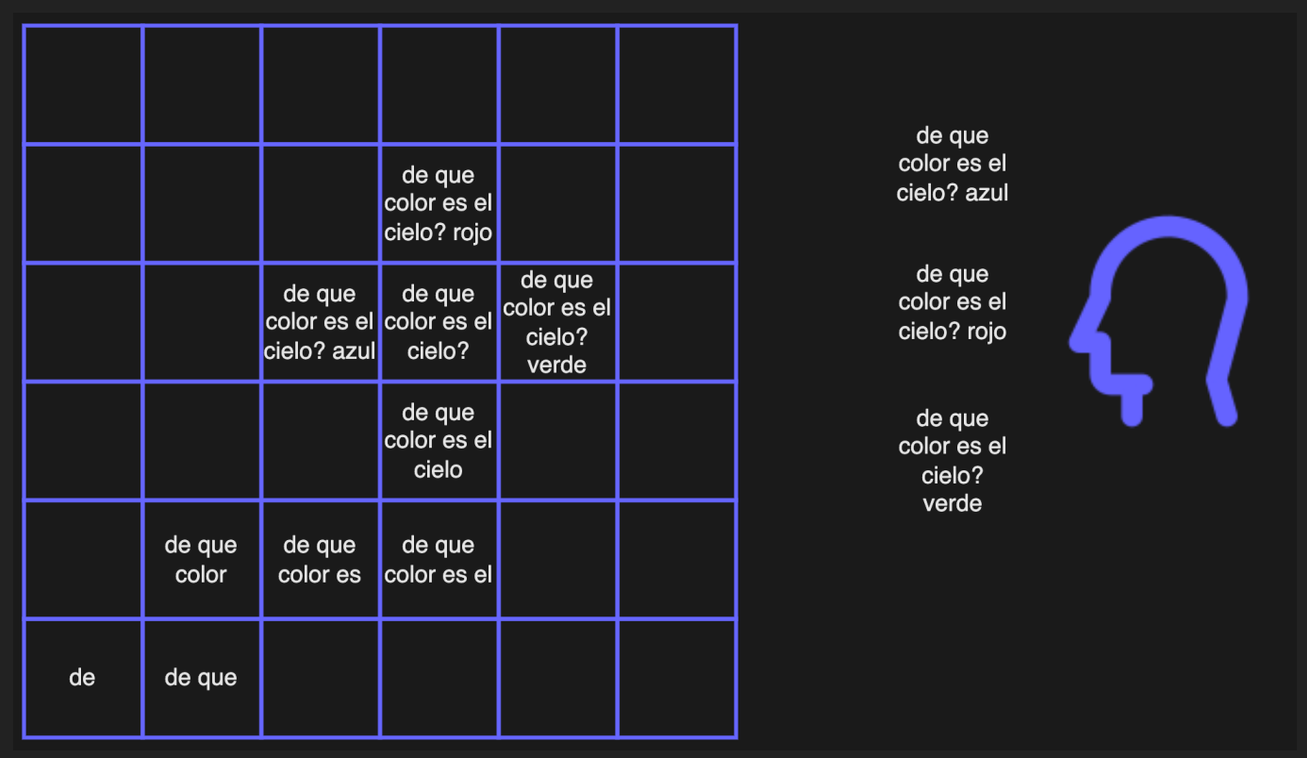
LOS 3 PASOS

- El modelo:
  - ✓ Aprende con ejemplos
  - ✓ Aprende preferencias
  - ✓ Se refuerza para que haga más de eso que es preferible.



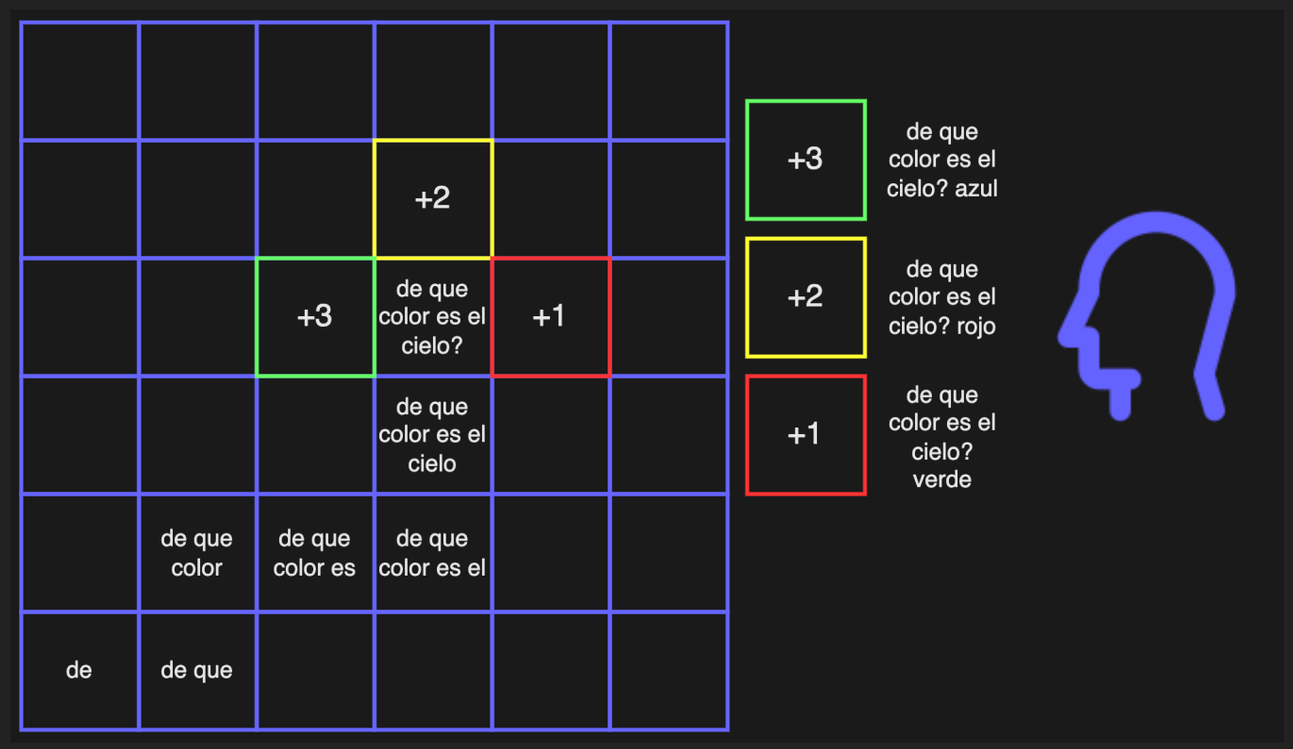
AJUSTE FINO SUPERVISADO (SFT)

- Ejemplo: Construcción de oraciones en un gridworld donde en cada casilla se agrega una palabra a la oración.
- ¿Cuál es la ultima palabra de la oración? La respuesta la escribe un Labeler (o anotador humano).



ENTRENAMIENTO DEL MODELO DE RECOMPENSA

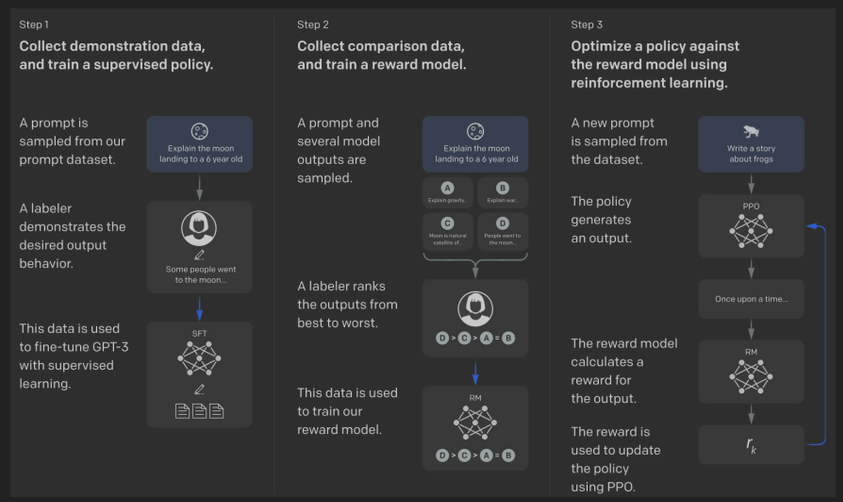
- Las respuestas escritas por los anotadores humanos ahora son rankeadas (calificadas) por el evaluador humano.
- Esto permite entrenar la red neuronal de recompensas (Reward Model).
- El RM es un modelo más pequeño que aprende a predecir la preferencia humana (es decir, qué tan "buena" es una respuesta) basándose en las clasificaciones humanas.



INSTRUCT-GPT

- InstructGPT es una versión de GPT-3 que fue entrenada para seguir instrucciones humanas de forma más útil y alineada.
- No solo genera texto, sino que interpreta y ejecuta mejor lo que el usuario quiere.
- InstructGPT es el modelo entrenado mediante RLHF.

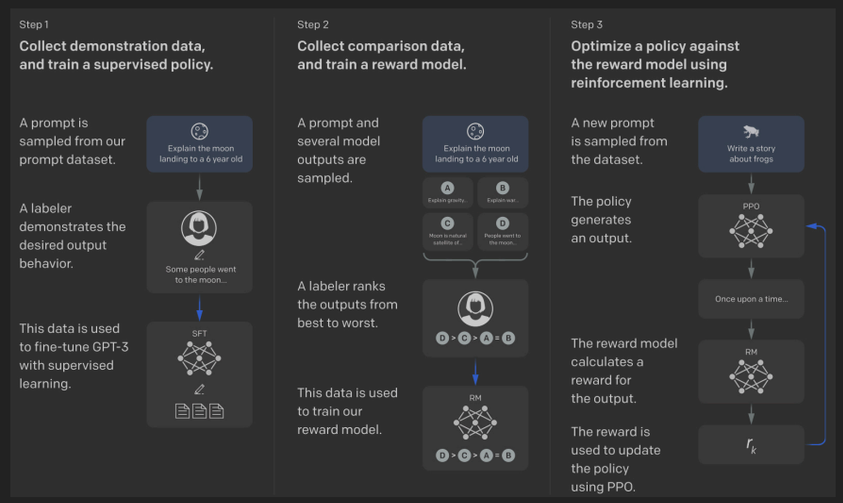
InstructGPT = GPT-3 + feedback humano + RLHF



INSTRUCT-GPT

- InstructGPT es una versión de GPT-3 que fue entrenada para seguir instrucciones humanas de forma más útil y alineada.
- No solo genera texto, sino que interpreta y ejecuta mejor lo que el usuario quiere.
- InstructGPT es el modelo entrenado mediante RLHF.

InstructGPT = GPT-3 + feedback humano + RLHF

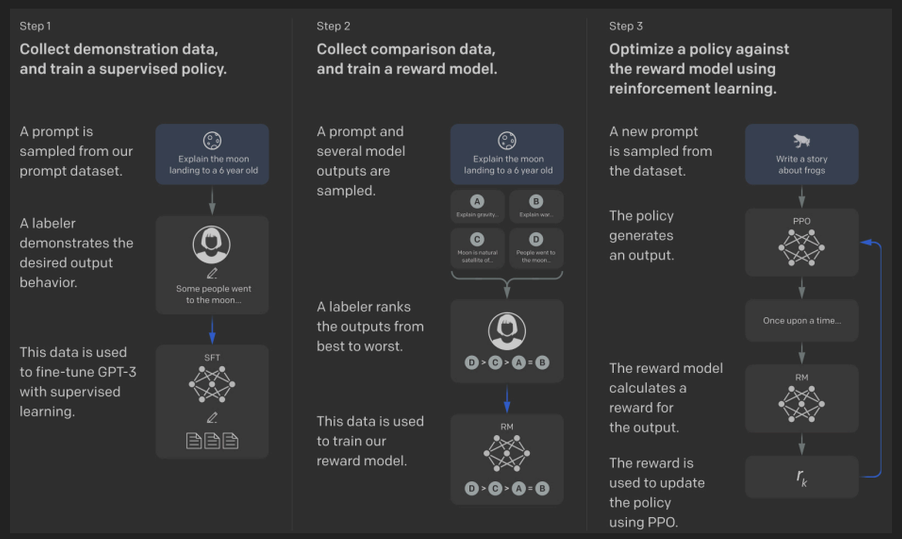


HALLUCINATION

- La Alucinación se presenta cuando el modelo genera información incorrecta o inventada, pero que parece plausible o verdadera.
- "El Everest está en África." ← Falso, pero dicho con seguridad.
- Hay dos tipos:

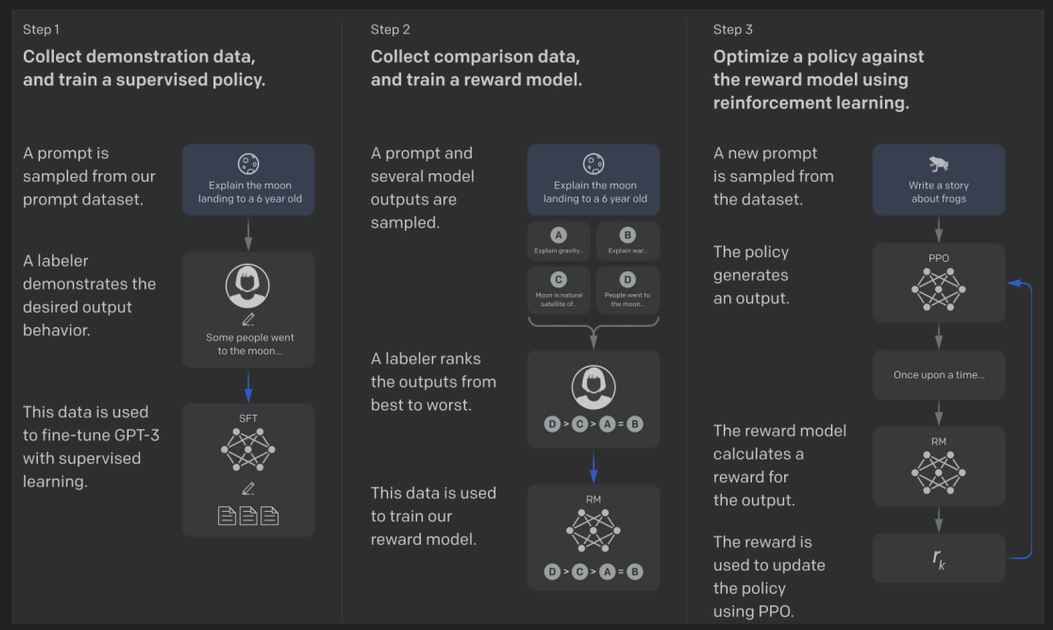
✓ Factuales: inventa datos (fechas, citas, hechos).

✓ Textuales: inventa partes de documentos, referencias, o citas falsas.



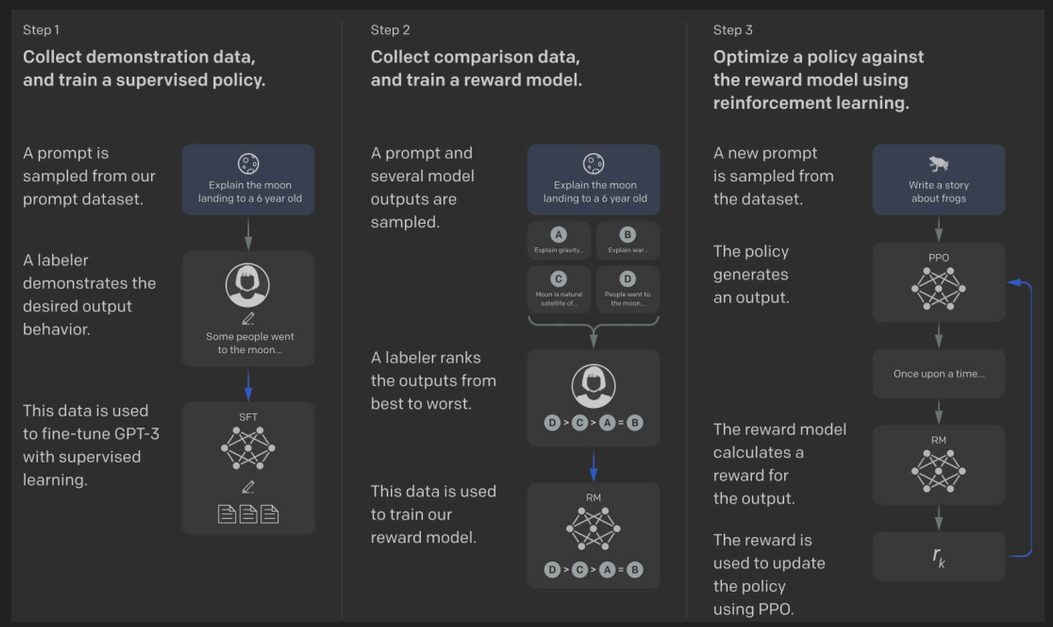
TOXICIDAD

- La Toxicidad existe cuando el modelo genera texto ofensivo, abusivo, violento, discriminatorio o que promueve el odio.
- Puede surgir por datos de entrenamiento contaminados con lenguaje tóxico.
- “Todas las personas de X grupo son [insulto].” ← Comentario tóxico.



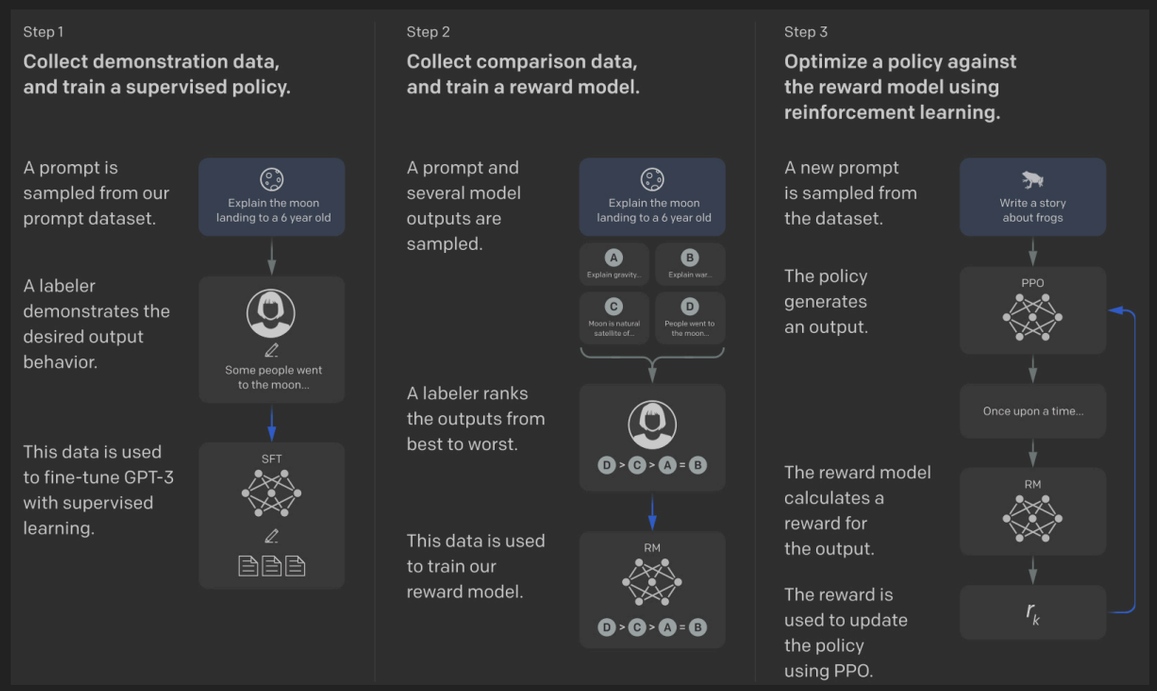
BIAS

- El Bias o sesgo ocurre cuando el modelo muestra preferencias, prejuicios o estereotipos de forma sistemática.
- Ejemplo: Si siempre asocia “criptomonedas” con estafas o “historia” con aburrido.
- Los sesgos en las respuestas reflejan (y a veces amplifican) los sesgos que ya están presentes en los datos de entrenamiento.



ALIGNMENT TAX

- Es el costo en desempeño que tiene el alinear un modelo con valores humanos.
- Un modelo más alineado (útil, seguro, ético) puede ser menos preciso o menos poderoso en ciertas tareas si ese alineamiento lo limita.
- Ejemplo: Un modelo alineado tal vez evita responder ciertas preguntas médicas por seguridad, mientras que uno no alineado si las responde (aunque pueda alucinar).



WINOGENDER

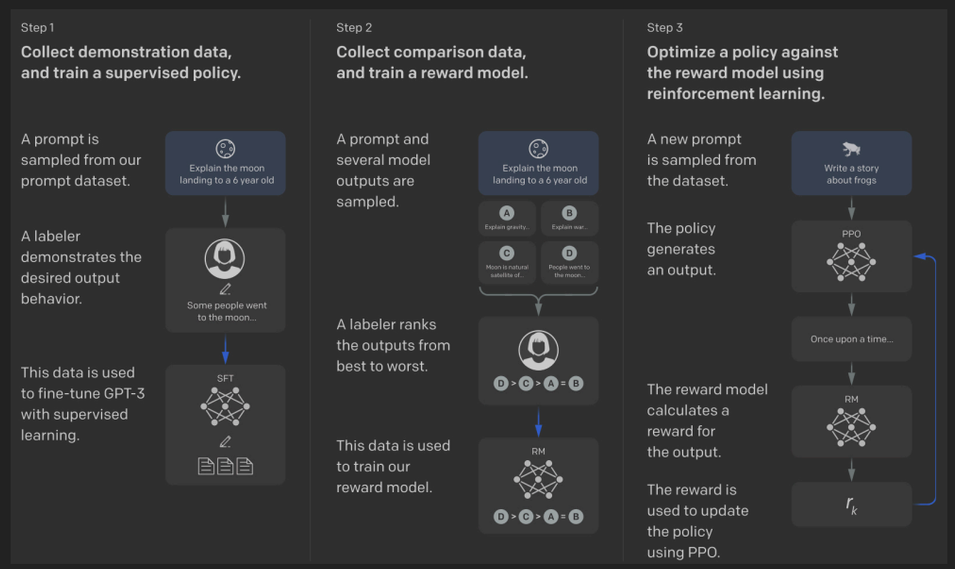
- Esta métrica prueba si el modelo asocia géneros con profesiones de forma estereotípica.
- Ejemplo:

"El médico habló con la paciente porque él estaba preocupado."

- El pronombre "él" se refiere claramente al médico.

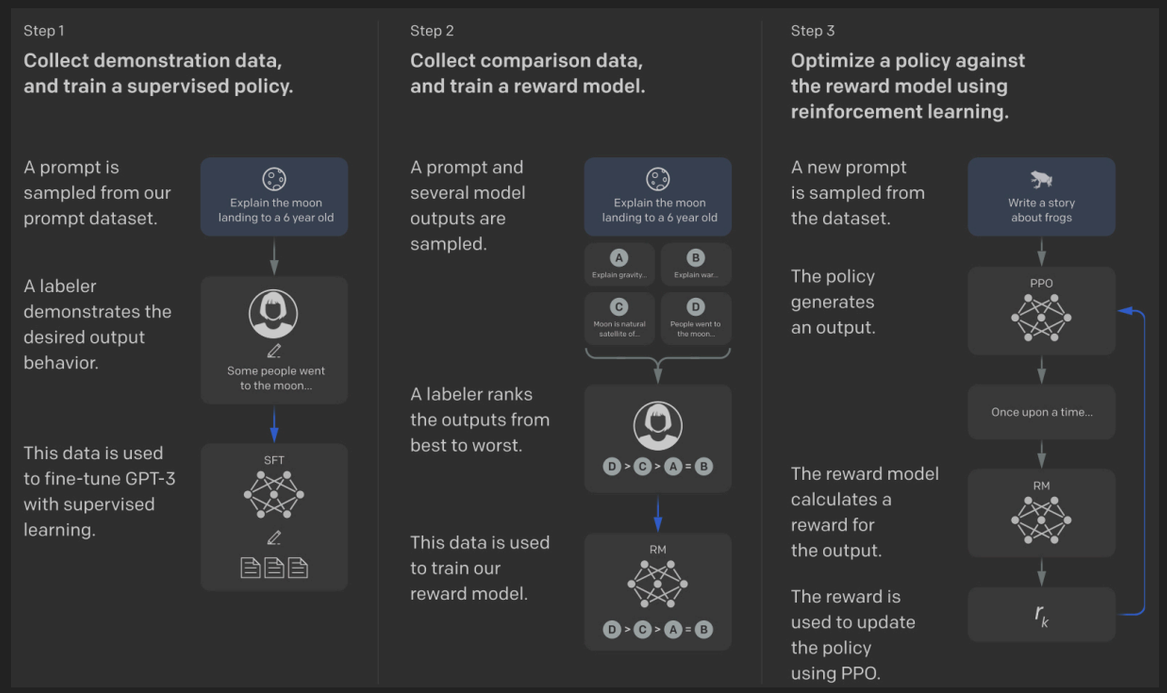
"El médico habló con la paciente porque ella estaba preocupada."

- "ella" debería referirse a la paciente, pero algunos modelos podrían pensar que "ella" se refiere al médico, si han aprendido un sesgo de que los médicos suelen ser mujeres (por ejemplo, por datos sesgados o estereotipos).



CROWS-PAIRS

- Es un dataset con pares de frases idénticas salvo por un sesgo (raza, religión, género…).
- “Los perros caniche toy son molestos” vs. “Los perros mestizos son molestos”.
- Mide si el modelo tiene preferencias implícitas por un grupo sobre otro.



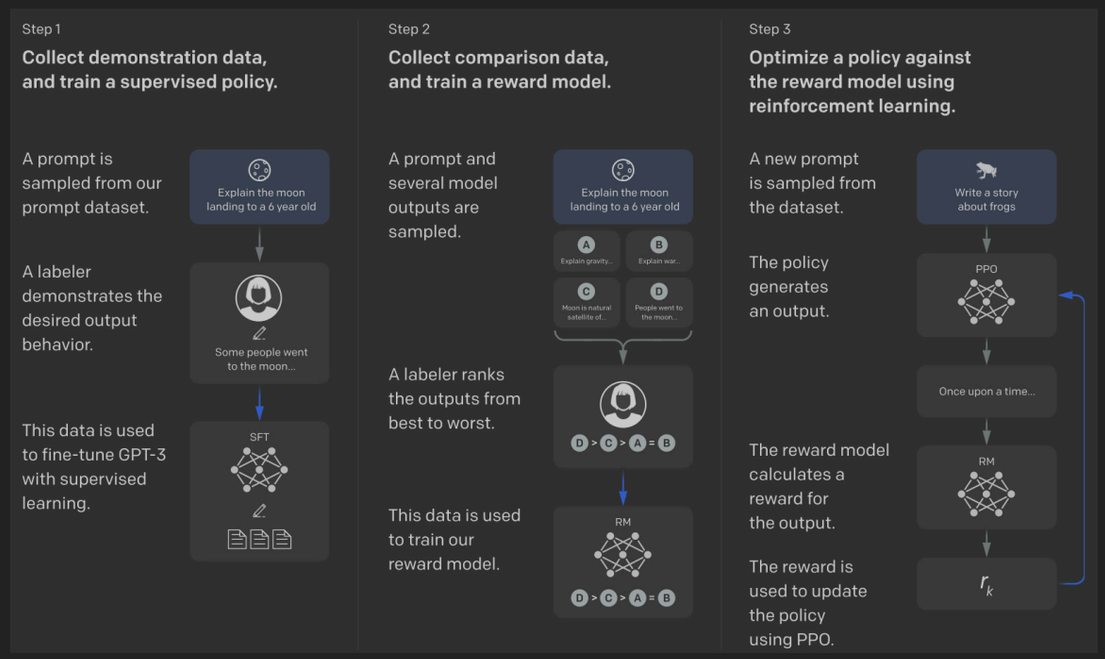
SQUAD (STANFORD QUESTION ANSWERING DATASET)

- Es un Benchmark de comprensión de lectura.
- El modelo debe leer un párrafo y responder preguntas cuya respuesta está explícita en el texto.
- Ejemplo:

Párrafo: “Marie Curie ganó dos premios Nobel…”

Pregunta: “¿Cuántos premios ganó Curie?”

- Mide comprensión de texto factual.



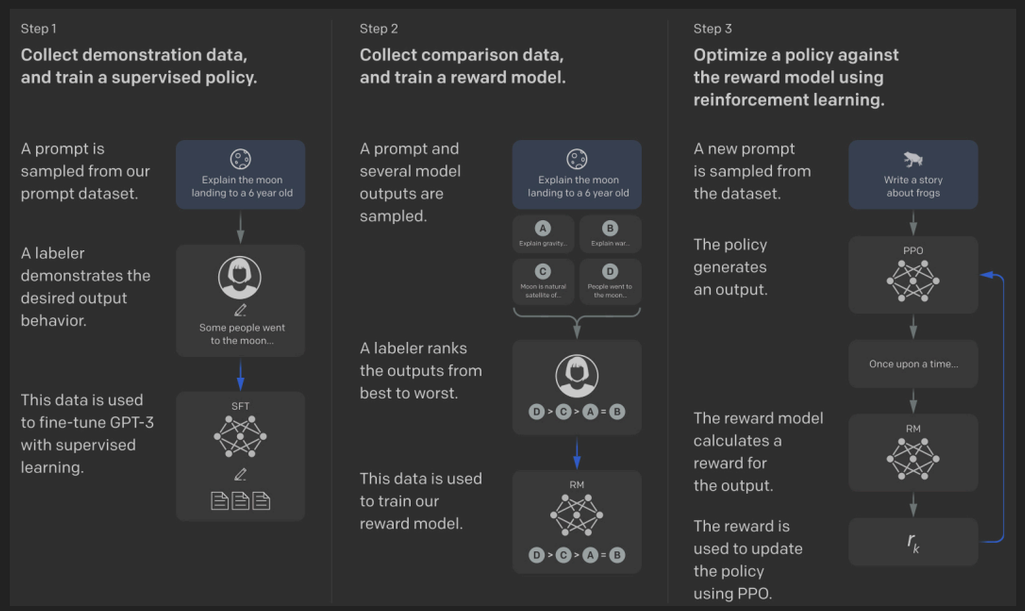
DROP (DISCRETE REASONING OVER PARAGRAPHS)

- El Razonamiento Discreto sobre Párrafos es una prueba en la que el modelo debe hacer razonamiento sobre el texto, no solo copiar.
- Ejemplo:

Párrafo: "Juan tuvo 5 manzanas, le dio 2 a Pedro y luego compró 4."

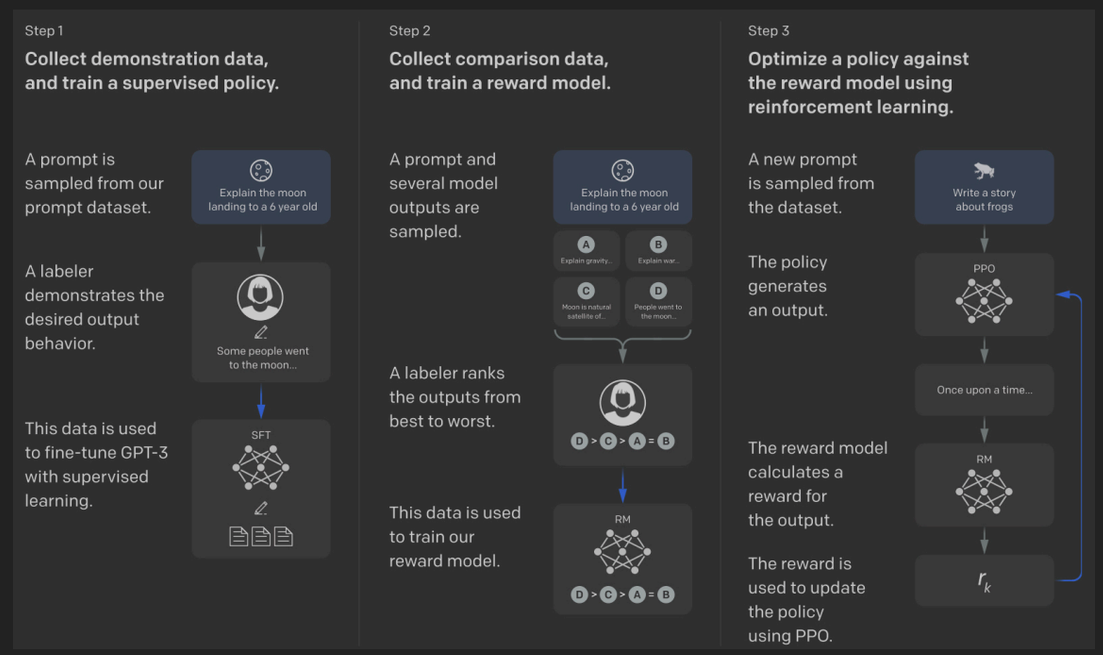
Pregunta: "¿Cuántas manzanas tiene Juan ahora?"

- Mide razonamiento lógico y aritmético en textos.



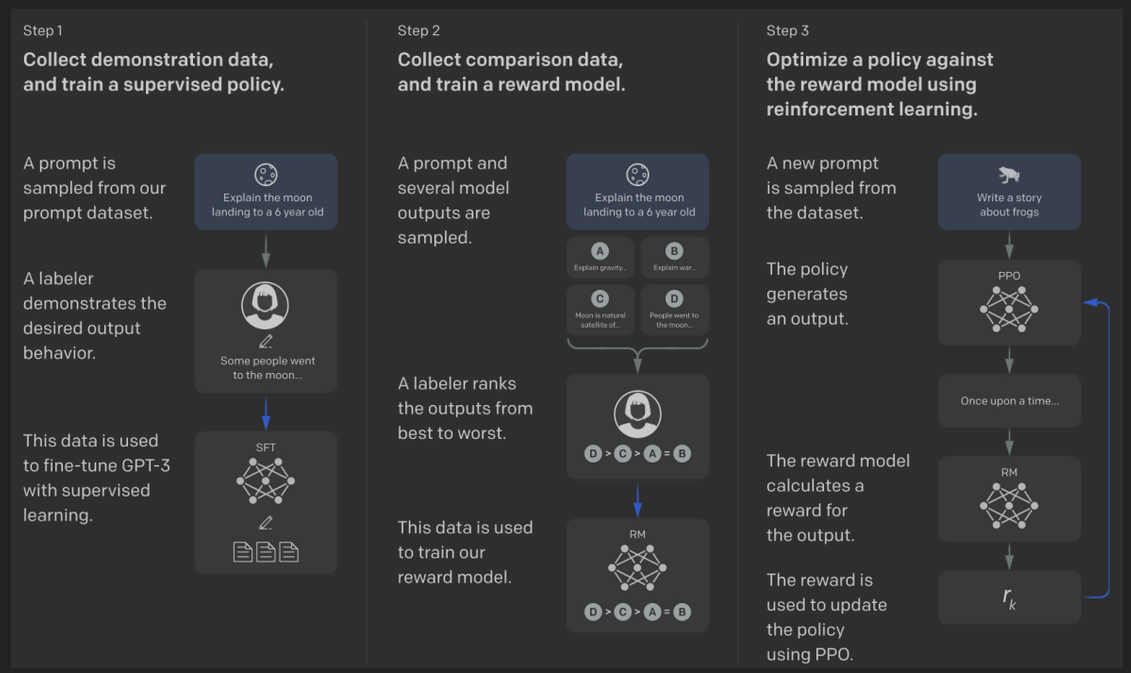
RESULTADOS RELEVANTES DEL METODO RLHF

- Al comparar estos modelos instructGPT con el GPT3 original, los evaluadores humanos prefirieron las respuestas de InstructGPT.
- El modelo IGPT pequeño (1.3 millones de parámetros) contra GPT3 original (175.000 millones de parámetros) fue el que alucinaba menos.



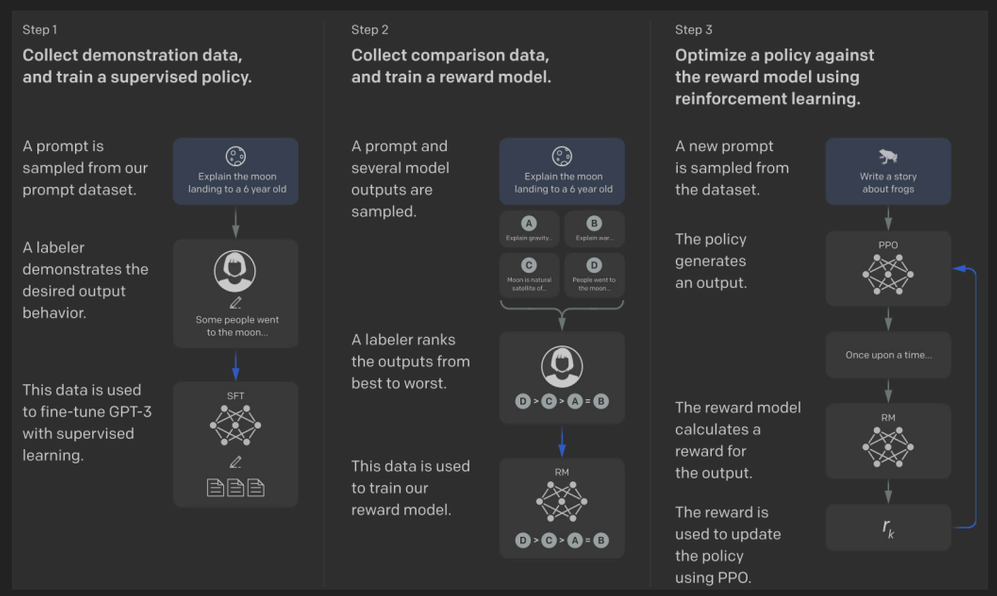
RESULTADOS RELEVANTES DEL METODO RLHF

- Toxicidad: Hubo una mejora modesta en toxicidad, el igpt generó un 25% menos de respuestas tóxicas que gpt3.
- Bias: usando métricas como Winogender (Rudinger et al., 2018) o CrowSPairs (Nangia et al., 2020) no hubo mejoras significativas.



RESULTADOS RELEVANTES DEL METODO RLHF

- Desventaja. Se detectó "Impuesto de alineamiento", el rendimiento bajó un poco en ciertas tareas académicas estándar en pruebas de comprension lectora como SQuAD (Rajpurkar et al., 2018) o DROP (Dua et al., 2019).
  - ✓ Al especializarse demasiado en seguir instrucciones perdió generalización en otras áreas.
  - ✓ Se dio una solución parcial mezclando PPO con datos del preentrenamiento de gpt3 original (variante ppo-ptx) sin afectar las preferencias de los humanos evaluadores.



EJEMPLO DE AFINACIÓN CON PPO

- Objetivo: Enseñar al modelo a responder de forma que incluya "hello" en sus respuestas.
- El ejemplo realiza lo siguiente:
  - ✓ Carga GPT-2 y lo adapta para RLHF (ValueHead)*.
  - ✓ Define recompensas: +1 si la respuesta contiene "hello", −1 si no.
  - ✓ Entrena con PPO:
    - ➡ Usa dos prompts.
    - ➡ Genera respuestas.
    - ➡ Calcula recompensas.
    - ➡ Ajusta el modelo con ppo_trainer.step().
  - ✓ Después del entrenamiento, genera texto con el modelo afinado.
  - ✓ *capa lineal que se conecta al final del modelo GPT y predice el "valor esperado" de la recompensa dada una entrada. Con ese valor calcula la Advantage y actualiza a GPT-2. (Head texto + Head valor).

CONFIGURACIÓN INICIAL

- Se prepara el tokenizador.
- Se carga el modelo GPT-2 y se adapta para PPO añadiendo una cabeza de valor (value head).

```
model_name = "gpt2"
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print(f"Usando dispositivo: {device}")

tokenizer = AutoTokenizer.from_pretrained(model_name)
if tokenizer.pad_token is None:
    tokenizer.pad_token = tokenizer.eos_token
tokenizer.padding_side = "left"

model = AutoModelForCausalLMWithValueHead.from_pretrained(model_name).to(device)
ref_model = create_reference_model(model).to(device)
```

CONFIGURACIÓN DE LA GENERACIÓN DE TEXTO

- max_new_tokens=30, # Máximo de tokens que puede generar (output)
- min_length=-1, # Largo mínimo del texto generado (-1 significa deshabilitado)
- do_sample=True, # Si True, activa la generación estocástica (no greedy)
- temperature=0.7, # Controla qué tan aleatoria es la generación (1 dn. Normal)
- top_k=0.0, # Top-k sampling (0 significa desactivado) elige los k tokens siguientes con mayor probabilidad
- top_p=1.0, # Top-p (nucleus sampling) (1.0 significa desactivado, sin limites) conj. más reducido de tokens con suma de prob ≥ p%.
- pad_token_id=tokenizer.pad_token_id, # Token para rellenar (padding), rellena las secuencias cortas
- eos_token_id=tokenizer.eos_token_id, # Token que indica fin de la secuencia con <eos>

<pad>

```
generation_config = GenerationConfig(
    max_new_tokens=30,
    min_length=-1,
    do_sample=True,
    temperature=0.7,
    top_k=0.0,
    top_p=1.0,
    pad_token_id=tokenizer.pad_token_id,
    eos_token_id=tokenizer.eos_token_id,
)
```

DEFINICIÓN DE LA RECOMPENSA

- Si el texto generado contiene "hello", recibe recompensa +1, si no, −1.
- Esto simula una retroalimentación humana (Human Feedback) mas precisamente simula a RM.

```
def reward_fn(texts_list):
    rewards = []
    for text in texts_list:
        if "hello" in text.lower():
            rewards.append(1.0)
        else:
            rewards.append(-1.0)
    return rewards
```

PREPARACIÓN DEL ENTRENAMIENTO

- Se crean dos prompts simples que se usarán como entrada al modelo.

```
prompts_text = ["Say something friendly:", "Introduce yourself:"]
dataset_dict = {"query": prompts_text}
```

ENTRENAMIENTO CON PPO

- Por cada prompt:
  - ✓ Se genera una respuesta.

```
response_full_from_generate = ppo_trainer.generate(
    current_query_1d, # Se pasa 1D
    return_prompt=True,
    **gen_kwargs_single
)
```

  - ✓ Se evalúa la recompensa con la función reward_fn.

```
rewards = reward_fn(generated_only_texts_for_reward)
```

  - ✓ Se actualiza el modelo usando ppo_trainer.step() para reforzar las respuestas buenas.

```
stats = ppo_trainer.step(queries_for_step, responses_generated_only_for_step, rewards_pt)
```

GENERACIÓN POST-ENTRENAMIENTO

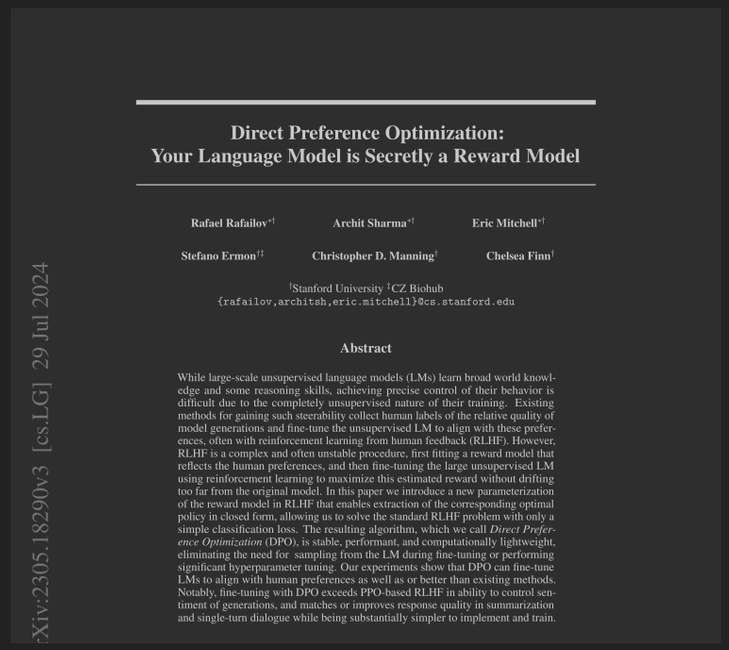
- Se vuelve a usar el modelo entrenado para generar nuevas respuestas y ver si aprendió a decir “hello".

```
for prompt_text_val in prompts_text:
    inputs = tokenizer(prompt_text_val, return_tensors="pt").to(device)
    gen_kwargs_val = generation_config.to_dict()
    if "attention_mask" in inputs:
        gen_kwargs_val["attention_mask"] = inputs["attention_mask"]

    response_ids = ppo_trainer.model.generate(
        input_ids=inputs["input_ids"],
        **gen_kwargs_val
    )
```

DPO

- DPO (Direct Preference Optimization) es una alternativa a RLHF que optimiza preferencias humanas directamente sin modelo de recompensa ni PPO (Rafailov et al., 2024).
- Es una técnica inspirada en RL, pero no usa una señal de recompensa explícita ni un entorno RL completo.
- En lugar de maximizar una recompensa a través de un entorno como en PPO, DPO ajusta directamente el modelo para que prefiera respuestas "mejores" sobre las "peores", basándose en comparaciones humanas.
- Es más estable, más directo y mas eficiente que RLHF con PPO.



Direct Preference Optimization:
Your Language Model is Secretly a Reward Model

Rafael Rafailov*† Archit Sharma*† Eric Mitchell*†
Stefano Ermon†‡ Christopher D. Manning† Chelsea Finn†
†Stanford University ‡CZ Biohub
{rafailov,architsh,eric.mitchell}@cs.stanford.edu

arXiv:2305.18290v3 [cs.LG] 29 Jul 2024

Abstract

While large-scale unsupervised language models (LMs) learn broad world knowledge and some reasoning skills, achieving precise control of their behavior is difficult due to the completely unsupervised nature of their training. Existing methods for gaining such steerability collect human labels of the relative quality of model generations and fine-tune the unsupervised LM to align with these preferences, often with reinforcement learning from human feedback (RLHF). However, RLHF is a complex and often unstable procedure, first fitting a reward model that reflects the human preferences, and then fine-tuning the large unsupervised LM using reinforcement learning to maximize this estimated reward without drifting too far from the original model. In this paper we introduce a new parameterization of the reward model in RLHF that enables extraction of the corresponding optimal policy in closed form, allowing us to solve the standard RLHF problem with only a simple classification loss. The resulting algorithm, which we call *Direct Preference Optimization* (DPO), is stable, performant, and computationally lightweight, eliminating the need for sampling from the LM during fine-tuning or performing significant hyperparameter tuning. Our experiments show that DPO can fine-tune LMs to align with human preferences as well as or better than existing methods. Notably, fine-tuning with DPO exceeds PPO-based RLHF in ability to control sentiment of generations, and matches or improves response quality in summarization and single-turn dialogue while being substantially simpler to implement and train.

RLM (MODELOS DE LENGUAJE DE RAZONAMIENTO)

- Los RLM (Besta et al., 2025) Son modelos que extienden los LLM con mecanismos de razonamiento avanzados.
- Algunos ejemplos de ellos: o1 y o3 de OpenAI, DeepSeek-V3 y QwQ.
- Combinan:
  - ✓ Aprendizaje por refuerzo (RL)
  - ✓ Heurísticas de búsqueda
  - ✓ LLMs.
- El RL en estos modelos:
  - ✓ Le permite al modelo explorar múltiples caminos de razonamiento.
  - ✓ Usa el mecanismo de prueba y error para encontrar cadenas de razonamiento óptimas.
- Estos modelos se entrenan usando:
  - ✓ PPO (Proximal Policy Optimization),
  - ✓ DPO (Direct Preference Optimization),
  - ✓ Y variantes como Reasoning Policy Optimization (RPO).

REFERENCIAS BIBLIOGRÁFICAS Y WEB (I)

- Ouyang, L., Wu, J., Jiang, X., Almeida, D., Wainwright, C., Mishkin, P., ... & Lowe, R. (2022). Training language models to follow instructions with human feedback. Advances in neural information processing systems, 35, 27730-27744.
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., ... & Polosukhin, I. (2017). Attention is all you need. Advances in neural information processing systems, 30.
- Rajpurkar, P., Jia, R., and Liang, P. (2018). Know what you don't know: Unanswerable questions for squad. arXiv preprint arXiv:1806.03822.
- Dua, D., Wang, Y., Dasigi, P., Stanovsky, G., Singh, S., and Gardner, M. (2019). Drop: A read- ing comprehension benchmark requiring discrete reasoning over paragraphs. arXiv preprint arXiv:1903.00161.
- Rudinger, R., Naradowsky, J., Leonard, B., and Van Durme, B. (2018). Gender bias in coreference resolution. In Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, New Orleans, Louisiana. Association for Computational Linguistics.
- Nangia, N., Vania, C., Bhalerao, R., and Bowman, S. R. (2020). CrowS-Pairs: A Challenge Dataset for Measuring Social Biases in Masked Language Models. In Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing, Online. Association for Computational Linguistics.
- Rafailov, R., Sharma, A., Mitchell, E., Manning, C. D., Ermon, S., & Finn, C. (2023). Direct preference optimization: Your language model is secretly a reward model. Advances in Neural Information Processing Systems, 36, 53728-53741.